

Automatisk kontrol af affugter



Indholdsfortegnelse

1. Indledning og problemformulering	3
Afgrænsning	3
1.1 Forretningsmål	4
1.2 Succeskriterier	4
1.3 Overordnede rammer / Betingelser	4
2. Kravspecifikation	5
2.1 Definitioner	5
2.2 Interessenter og aktører	5
2.3 Funktionelle krav	6
2.3.1 Overordnede funktioner	6
2.3.2 Detaljerede krav	7
2.4 Use cases / scenarier	8
2.5 Prioriteret regelrækkefølge	15
2.6.1 Ikke-funktionelle krav	17
2.6.2 Datakrav	18
2.6.3 Integrationer og afhængigheder	18
2.7 Prioritering	18
2.7.1 Sikkerhedstilstand og failure modes	19
3. Brugervejledning	21
3.1 Hardware-opsætning	21
3.2 Installation	23
3.2.1 Systempakker	23
3.2.2 Hent kildekoden og installer Python-pakker	24
3.2.3 Database	25
3.2.4 Konfigurationsfilen .env	25
3.2.5 Start som tjeneste	25
3.3 Anvendelse af websiden	26
3.3.1 Dashboard	27
3.3.2 Aktivitetslog	28
3.3.3 Error log	29
3.3.4 Debug	30
4. Teknisk dokumentation	32
4.1 Arkitektur (API-oversigt)	33
4.2 Procesdiagrammer (opstart, styringsløkke, determine-state)	35
4.3 API-beskrivelse	38

4.3.1 Systemendpoints	39
4.3.2 Dashboard-endpoints.....	39
4.3.3 Test- og fejlfindingsendpoints	39
4.4 Database.....	40
4.5 Biblioteker og teknologivalg	41
5. Testrapport	42
5.1 Accepttest.....	42
6. Konklusion	47
7. Bilag	48

1. Indledning og problemformulering

En affugter kan være nødvendig for at holde luftfugtigheden i et rum på et acceptabelt niveau, men kan også være dyr at holde i drift.

Projektets overordnede problemstilling er derfor:

Hvordan kan vi udvikle et automatiseret system, som styrer en affugter ud fra både den aktuelle luftfugtighed og varierende elpriser, så affugteren primært kører i døgnets billigste perioder, og så den tændes uanset elpris, når luftfugtigheden overstiger en fastsat kritisk grænseværdi, undtagen i foruddefinerede tidsrum, hvor drift er begrænset?

For at besvare problemformuleringen undersøges følgende delproblemer:

- Hvordan kan luftfugtigheden måles og anvendes som grundlag for automatisk styring?
- Hvordan kan aktuelle elpriser indhentes og anvendes til dynamisk at afgøre, hvornår det er økonomisk fordelagtigt at køre affugteren?
- Hvordan kan systemet prioritere luftfugtighed over elpris, når luftfugtigheden overstiger en fastsat kritisk grænseværdi?
- Hvordan kan affugterens kompressor beskyttes mod genstart inden for et fastsat minimumsinterval?
- Hvordan kan systemet fungere som en selvstændig lokal løsning uden afhængighed af en ekstern server?
- Hvordan bør systemet håndtere fejl såsom manglende internetforbindelse, utilgængelige elprisdata, kommunikationsfejl og hardwarefejl?

Afgrænsning

Projektets primære fokus er udviklingen af den automatiske styring af affugteren. Systemets minimumsløsning består af en mikrocontroller, en luftfugtighedssensor og en smart plug samt den nødvendige software til indhentning af elpriser og styring af affugteren.

Produktet skal i sidste ende kunne leveres som en enkelt enhed, som læser elpriser, måler luftfugtighed, og derudfra automatisk styrer en tilsluttet smart plug.

1.1 Forretningsmål

Brugeren ønsker at undgå skimmel i værelset ved at holde luftfugtigheden nede og samtidigt reducere udgiften til at drive affugteren. I dag kører affugteren typisk i timerne efter kl.17.00, hvor elprisen er højest, og som er mindst to til tre gange dyrere end døgnets billigste timer. Målet er derfor at flytte affugterens drift til døgnets billigste kvarterer uden at gå på kompromis med luftfugtigheden.

1.2 Succeskriterier

Projektet betragtes som vellykket, når:

1. luftfugtigheden uden for dark time højst overstiger den kritiske grænse i 60 minutter ad gangen,
2. affugteren ved normal drift kun kører i døgnets billige kvarterer uden for dark time,
3. systemet kører i mindst [72] timer uden menneskelig indgriben,
4. alle krav med prioriteten Must (afsnit 2.7) er opfyldt efter acceptkriterierne i afsnit 5.1.

1.3 Overordnede rammer / Betingelser

Forudsætning: Affugteren (eeese Emil 2508) har ingen netværksfunktion og styres derfor udelukkende ved at afbryde og genetablere strømmen via smart plug'en. Løsningen forudsætter, at affugteren automatisk genoptager driften, når strømmen tilsluttes igen. Ifølge brugsanvisningen kører affugteren i samme tilstand som før strømafbrydelsen (bilag 2, s. 12), og egenskaben er afprøvet af gruppen.

2. Kravspecifikation

2.1 Definitioner

Akronym	Definition
RH	Relativ luftfugtighed i procent.
Zigbee-sensor	Den trådløse Zigbee-sensor, som er placeret i lokalet.
DHT11-sensor	DHT11-sensoren, som er tilsluttet Raspberry Pi'ens GPIO-pins.
Gyldig måling	En måling inden for 0–100 % RH, som ikke er ældre en sensorens gyldighedsperiode: 30 minutter for Zigbee-sensoren og 3 minutter for DHT11-sensoren.
Styringscyklus	Systemets beslutningsinterval. Systemet træffer en ny beslutning hvert 15. minut.
Kvarterpris	Elprisen for et kvarter (15 minutter). Døgnet har 96 kvarterpriser.
Prisgrænse	Den pris, som afgrænser de billigste 25 % af døgnets 96 kvarterpriser.
Billigt kvarter	Et kvarter, hvor elprisen er på eller under dagens prisgrænse.
Slukgrænse	Den luftfugtighed (45% RH), under hvilken affugteren slukkes.
Kritisk grænse	Den luftfugtighed (70% RH), ved og over hvilken affugteren tændes uanset elpris.
Minimumsinterval	Den mindste tid (15 minutter) fra en slukning, til affugteren må tændes igen.
Dark time	Det foruddefinerede tidsrum kl. 22.00–06.00, hvor affugteren er slukket.
Sikkerhedstilstand	Tilstand, når der ikke findes gyldige elpriser, heller ikke gemte. Affugteren styres udelukkende efter luftfugtigheden: den tændes ved den kritiske grænse og slukkes ved slukgrænsen (K-011).
API	Programmeringsgrænseflade, som gør det muligt for systemer at udveksle data.
MQTT	Protokol til beskeder mellem enheder, anvendt mellem Zigbee2MQTT og systemet.
GPIO	Raspberry Pi'ens tilslutningsben til elektroniske komponenter.

2.2 Interessenter og aktører

Rolle	Beskrivelse	Relaterede krav
Bruger (husstanden)	Følger systemets tilstand og aktive fejl på websiden.	K-016
Administrator (gruppen)	Installerer systemet, fastsætter grænseværdier og vedligeholder softwaren.	K-004, K-005, K-014
Systemet interagerer med følgende aktører:		
Aktør	Type	Rolle i systemet
Bruger	Person	Aflæser systemets tilstand på websiden
Luftfugtighedssensor, (Zigbee eller DHT11)	Hardware	Leverer målinger af luftfugtigheden. Sensortypen vælges i konfigurationsfilen (K-001)
Elpris-API	Ekstern tjeneste	Leverer dagens kvarterpriser (K-003)
Smart plug (Shelly)	Hardware	Tænder og slukker strømmen til affugteren (K-007)
Affugter (eeese Emil 2508)	Apparat	Sænker luftfugtigheden, når den får strøm

2.3 Funktionelle krav

2.3.1 Overordnede funktioner

Systemet styrer affugteren automatisk og sørger for, at strømforbruget så vidt muligt placeres i døgnets billigste kvarterer. Styringsmodellen anvender både den aktuelle luftfugtighed i lokalet og den aktuelle elpris for at afgøre, om smart plug'en skal være tændt eller slukket, og styrer på denne måde affugteren.

Systemet understøtter to sensortyper: en trådløs Zigbee-sensor og en kablet DHT11-sensor tilsluttet Raspberry Pi's GPIO-pins. Hvilken sensor der anvendes, vælges ved opsætningen i konfigurationsfilen (.env). Zigbee-sensoren kan placeres frit i lokalet, fx tæt på de hjørner hvor skimlen opstår, mens DHT11-sensoren er uafhængig af batteri og radioforbindelse. Systemet anvender den seneste gyldige måling uanset kilde og understøtter dermed begge opsætninger uden ændringer i programkoden.

Når luftfugtigheden er over slukgrænsen, og det aktuelle kvarter er billigt (se K-002), tændes affugteren. Den kører, indtil luftfugtigheden er ved eller under slukgrænsen, eller kvarteret ikke længere er billigt. Hvis luftfugtigheden når den kritiske grænse, tændes affugteren uanset elpris. Når luftfugtigheden igen er under den kritiske grænse, gælder de almindelige regler, og driften kan derfor blive afbrudt, hvis kvarteret ikke er billigt. Findes der ingen gyldige elpriser, går systemet i sikkerhedstilstand, hvor affugteren udelukkende styres efter luftfugtigheden (K-011). I dark time er affugteren altid slukket (K-018).

For at beskytte kompressoren må affugteren tidligst tændes 15 minutter efter sidste slukning (se K-006). Ved normal slukning på tænd/sluk-knappen kører blæseren videre i nogle sekunder for at fjerne den varme luft inde i enheden (bilag 2, s. 9). Når strømmen afbrydes via smart plug'en, sker dette ikke. Brugsanvisningen angiver ikke et minimumsinterval, mens andre eese-affugtere angiver ca. 3 minutter, før kompressoren arbejder igen. Gruppen har derfor valgt 15 minutter som sikkerhedsmargin. Minimumsintervallet gælder i alle driftstilstande, også ved kritisk luftfugtighed og ved fejl.

Systemet anvender tænd/sluk-regulering og ikke en graderet regulering (fx PID). Gruppen har valgt ikke at indføre en separat, højere tændgrænse, fordi affugteren ved normal drift kun kører i døgnets billigste kvarterer. Minimumsintervallet på 15 minutter (K-006) begrænser, hvor ofte affugteren kan tændes og slukkes.

Affugteren kan kun tændes og slukkes via smart plug'en, og luftfugtigheden kan derfor overstige den kritiske grænse i tre situationer:

1. Styringscyklussen: systemet træffer en beslutning hvert 15. minut, så der kan gå op til 15 minutter, før systemet reagerer.

2. Minimumsintervallet: affugteren må tidligst tændes 15 minutter efter sidste slukning (K-006). Luftfugtigheden kan derfor kortvarigt overstige den kritiske grænse, før affugteren tændes.

3. Dark time (K-018): i disse tidsrum er driften begrænset, også når luftfugtigheden overstiger den kritiske grænse. Luftfugtigheden kan derfor overstige den kritiske grænse i hele tidsrummet og sænkes først, når tidsrummet slutter.

Denne prioritering er valgt af brugeren af hensyn til støj i soveværelset.

Hvordan reglerne spiller sammen, fremgår af afsnit 2.5.

2.3.2 Detaljerede krav

Krav-ID	Krav	Kilde
K-001	Systemet skal anvende den seneste gyldige måling af luftfugtigheden fra den konfigurerede sensor. En måling er gyldig i 30 minutter fra Zigbee-sensoren og i 3 minutter fra DHT11-sensoren.	Gruppen
K-002	Systemet skal én gang i døgnet beregne en prisgrænse ud fra døgnets 96 kvarterpriser, så de billigste 25 % (24 til 6 timer; brugercasen viser, at kvarterer) har en pris på eller under prisgrænsen.	Gruppens valg. 24 kvarterer svarer til 6 timer; brugercasen viser, at affugteren skal køre i over 3 timer.
K-004	Affugteren skal tændes, når luftfugtigheden er over slukgrænsen på 45% RH, og det aktuelle kvarter er billigt (se K-002). En tændt affugter skal køre videre, indtil luftfugtigheden er under slukgrænsen på 45% RH, eller det aktuelle kvarter ikke længere er billigt.	Casen / brugeren
K-005	Når luftfugtigheden overstiger den kritiske grænse på 70% RH, skal affugteren tændes uanset elpris. Når	Casen: skimmel opstår ved luftfugtighed over 68 %

	luftfugtigheden igen er under den kritiske grænse, gælder de almindelige regler (K-004). Dette gælder ikke i dark time (se K-018).	
K-006	Affugteren må ikke tændes igen, før der er gået mindst 15 minutter siden sidste slukning. Minimumsintervallet gælder i alle driftstilstande, også ved kritisk luftfugtighed og ved fejl.	Gruppens valg på baggrund af brugsanvisningen (bilag 2, s. 9) og andre eese-affugtere
K-016	Systemet skal på en lokal webside vise aktuel luftfugtighed og datakilde, om affugteren er tændt eller slukket, dagens prisgrænse og aktuel elpris, historik over målinger og tænd/sluk-hændelser samt loggede fejl.	Brugeren
K-018	Affugteren må ikke være tændt mellem kl. 22.00 og 06.00 (dark time), heller ikke når luftfugtigheden overstiger den kritiske grænse. Kører affugteren kl. 22.00, slukkes den. K-018 har forrang for alle regler, der tænder affugteren (K-004, K-005, K-011).	Brugeren: støj i soveværelset

Acceptkriterierne for de enkelte krav fremgår af afsnit 5.1.

2.4 Use cases / scenarier

Use case 1.0 - 8.0 udløses af systemet selv, ikke af en bruger. Kun use case 9.0 har en menneskelig aktør.

Use Case	Opstart af systemet
ID	1.0
Aktør(er)	Systemet(primær), systemd, smart plug, elpris-API
Trigger	Raspberry Pi'en startes, og systemd starter applikationen
Prækondition	Hardwaren er tilsluttet, og .env er konfigureret
Postkondition	Systemet kører og træffer beslutninger hvert 15. minut
Beskrivelse	Systemet klargør sig selv ved start af Raspberry Pi'en. Det kontrollerer forbindelsen til smart plug'en, henter dagens elpriser, beregner prisgrænsen og starter dataindsamlingen og styringsløkken. Opstarten sker automatisk og kræves ingen handling fra brugeren.
Normalt forløb	1. Applikationen starter 2. Forbindelsen til smart plug'en kontrolleres 3. Dagens elpriser hentes og gemmes 4. Prisgrænsen beregnes

	5. Systemet starter dataindsamlingen fra den konfigurerede sensor: enten en MQTT-lytter til Zigbee-sensoren eller en løkke, der aflæser DHT11-sensoren hvert 30. sekund 6. Styringsløkken starter og venter på næste kvarter
Alternativt forløb	2.1 Smart plug'en svarer ikke: fejlen logges, systemet fortsætter 3.1 Elpriserne kan ikke hentes: fejlen logges, systemet starter i sikkerhedstilstand (se 6.0)
Udvidelsesmuligheder	
Relaterede krav	K-002, K-003, K-007, K-015

Use Case	Dagens elpriser hentes
ID	2.0
Aktør(er)	Systemet (primær), elpris-API
Trigger	Styringscyklussen finder ingen pris for det aktuelle kvarter i databasen, eller konstaterer, at dagens prisgrænse er fra i går. Sker typisk når døgnet skifter eller ved opstart (se 1.0)
Prækondition	Systemet kører og har internetforbindelse
Postkondition	Dagens 96 kvarterpriser er gemt i databasen, og dagens prisgrænse er beregnet
Beskrivelse	Én gang i døgnet henter systemet dagens kvarterpriser og beregner en prisgrænse, som afgrænser de billigste 25 % af døgnets kvarterer. Prisgrænsen bruges resten af døgnet til at afgøre, om et kvarter er billigt.
Normalt forløb	1. Styringscyklussen slår prisen op for det aktuelle kvarter i databasen 2. Findes der ingen pris, henter systemet kvarterpriser for i dag og i morgen fra elpris-API'et 3. Priserne gemmes i databasen. Allerede gemte kvarterer springes over 4. Systemet konstaterer, at prisgrænsen er fra i går, og beregner dagens prisgrænse som den 24. laveste af døgnets 96 priser 5. Prisgrænsen anvendes resten af døgnet og vises på websiden
Alternativt forløb	2.1 Prisen for det aktuelle kvarter findes allerede i databasen: trin 2 og 3 springes over 2.2 API'et svarer ikke, eller der modtages færre end 96 priser: fejlen logges, og systemet kører videre i sikkerhedstilstand (se 6.0), indtil priserne kan hentes 4.1 Prisgrænsen er allerede beregnet for i dag: trinnet springes over 4.2 Prisgrænsen kan ikke beregnes: systemet anvender ikke gårsdagens

	prisgrænse, men går i sikkerhedstilstand (se 6.0)
Udvidelsesmuligheder	
Relaterede krav	K-002, K-003, K-008, K-011
Use Case	Normal drift i billigt kvarter
ID	3.0
Aktør(er)	Systemet (primær), luftfugtighedssensor, smart plug
Trigger	Styringscyklussen udløses (hvert 15. minut)
Prækondition	Affugteren er slukket, minimumsintervallet er gået, der findes en gyldig måling og en gyldig prisgrænse, og det er ikke dark time
Postkondition	Affugteren kører i et eller flere billige kvarterer, og tænd- og sluk-hændelserne er gemt i databasen med begrundelse
Beskrivelse	Systemet sænker luftfugtigheden og strømmen er brugt i de billige kvarterer.
Normalt forløb	1. Systemet henter den seneste gyldige måling og konstaterer, at luftfugtigheden er over slukgrænsen 2. Systemet henter prisen for det aktuelle kvarter og konstaterer, at kvarteret er billigt 3. Systemet tænder smart plug'en og gemmer tænd-hændelsen med begrundelse 4. Ved hver efterfølgende styringscyklus kontrolleres luftfugtigheden og prisen på ny 5. Luftfugtigheden når slukgrænsen, og systemet slukker smart plug'en og gemmer sluk-hændelsen
Alternativt forløb	2.1 Kvarteret er ikke billigt: affugteren forbliver slukket 5.1 Kvarteret er ikke længere billigt, før slukgrænsen er nået: affugteren slukkes, og driften genoptages først, når luftfugtigheden igen er over slukgrænsen i et billigt kvarter 5.2 Luftfugtigheden når den kritiske grænse: affugteren fortsætter uanset elpris (se 4.0) 5.3 Klokken bliver 22.00: affugteren slukkes (se 5.0)
Udvidelsesmuligheder	
Relaterede krav	K-001, K-002, K-004, K-006, K-009

Use Case	Drift ved kritisk høj luftfugtighed
ID	4.0
Aktør(er)	Systemet (primær), luftfugtighedssensor, smart plug
Trigger	Styringscyklussen udløses (hvert 15. minut)
Prækondition	Affugteren er slukket, minimumsintervallet er gået, der findes en gyldig måling, og det er ikke dark time
Postkondition	Affugteren har kørt uanset elpris, og tænd- og sluk- hændelserne er gemt i databasen med begrundelse
Beskrivelse	Når luftfugtigheden når den kritiske grænse, prioriteres luftfugtigheden over elprisen, og affugteren tændes uanset, om kvarteret er billigt.
Normalt forløb	1. Systemet henter den seneste gyldige måling og konstaterer, at luftfugtigheden er over kritisk grænsen (70% RH eller højere) 2. Systemet tænder smart plug'en og gemmer tænd-hændelsen med begrundelse 3. Ved hver efterfølgende styringscyklus kontrolleres luftfugtigheden og prisen på ny 4. Luftfugtigheden falder under den kritiske grænsen, nu kører systemet igen efter de almindelige regler (se 3.0)
Alternativt forløb	1.1 Minimumsintervallet er ikke gået: affugteren forbliver slukket, indtil de 15 minutter er gået, hvorefter den tændes 4.1 Kvarteret er ikke billigt, når luftfugtigheden er under den kritiske grænse: affugteren slukkes, selvom slukgrænsen ikke er nået 4.2 Kvarteret er billigt: affugteren fortsætter, indtil luftfugtigheden når slukgrænsen (se 3.0) 4.3 Klokken bliver 22.00: affugteren slukkes, også selvom luftfugtigheden stadig er kritisk (se 5.0)
Udvidelsesmuligheder	
Relaterede krav	K-001, K-005, K-006, K-009, K-018

Use Case	Dark time
ID	5.0
Aktør(er)	System (primær) ,luftfugtighedssensor, smart plug
Trigger	Styringscyklussen udløses kl. 22.00
Prækondition	Affugteren kører, og klokken er mellem 06.00 og 22.00

Postkondition	Affugteren har været slukket i hele tidsrummet kl. 22.00–06.00, uanset luftfugtigheden, og sluk-hændelsen er gemt i databasen
Beskrivelse	Af hensyn til støj i soveværelset må affugteren ikke køre om natten. I tidsrummet kl. 22.00–06.00 er affugteren derfor slukket, også når luftfugtigheden overstiger den kritiske grænse
Normalt forløb	1. Styringscyklussen kl. 22.00 konstaterer, at dark time er begyndt 2. Systemet slukker smart plug'en og gemmer sluk-hændelsen med begrundelse 3. Ved hver efterfølgende styringscyklus konstaterer systemet, at dark time stadig gælder, og affugteren forbliver slukket 4. Luftfugtigheden stiger over den kritiske grænse kl. 23.00, men affugteren forbliver slukket 5. Kl. 06.00 gælder de almindelige regler igen, og affugteren tændes, hvis luftfugtigheden er over den kritiske grænse (se 4.0) eller kvarteret er billigt (se 3.0).
Alternativt forløb	1.1 Affugteren er allerede slukket kl. 22.00: den forbliver slukket, og ingen hændelse gemmes 5.1 Luftfugtigheden er under slukgrænsen kl. 06.00: affugteren forbliver slukket 5.2 Minimumsintervallet er ikke gået kl. 06.00: affugteren tændes først, når de 15 minutter er gået
Udvidelsesmuligheder	At brugeren selv kan ændre tidsrummet for dark time i konfigurationen
Relaterede krav	K-005, K-006, K-009, K-018

Use Case	Drift i sikkerhedstilstand
ID	6.0
Aktør(er)	System (primær) ,luftfugtighedssensor, smart plug
Trigger	Styringscyklussen udløses, og der findes ingen gyldige elpriser
Prækondition	Sensoren leverer gyldige målinger, elpriser findes ikke i databasen, og kan hellere ikke hentes fra API'et, minimumsintervallet er gået , og det er ikke dark time
Postkondition	Affugteren er styret udelukkende efter luftfugtigheden, og tænd- og sluk-hændelserne er gemt i databasen med begrundelse
Beskrivelse	Når elpriserne ikke er tilgængelige, går systemet i sikkerhedstilstanden, hvor beslutningerne udelukkende træffes ud fra den kritisk grænse og slukgrænsen.
Normalt forløb	1. Systemet konstaterer, at der hverken findes gemte elpriser eller priser,

	der kan hentes (se 2.0) 2. Sikkerhedstilstand aktiveres, fejlen logges, og tilstanden vises på websiden 3. Luftfugtigheden når 70 % RH, og systemet tænder smart plug'en uanset elpris 4. Luftfugtigheden falder til 45 % RH, og systemet slukker smart plug'en 5. Elpriserne kan igen hentes, systemet forlader sikkerhedstilstanden, og de almindelige regler gælder (se 3.0)
Alternativt forløb	3.1 Luftfugtigheden er mellem 45 % og 70 % RH: affugteren forbliver slukket, så længe sikkerhedstilstanden varer 3.2 Klokken bliver 22.00: affugteren forbliver slukket, også ved kritisk luftfugtighed (se 5.0) 4.1 Sensoren holder op med at levere gyldige målinger: affugteren slukkes (se 7.0)
Udvidelsesmuligheder	At systemet anvender gennemsnittet af de seneste dages priser som midlertidig prisgrænse i stedet for helt at se bort fra elprisen
Relaterede krav	K-003, K-008, K-011, K-005, K-009, K-016

Use Case	Sensorfejl
ID	7.0
Aktør(er)	System (primær) ,luftfugtighedssensor
Trigger	Styringscyklussen udløses, og der findes ingen gyldig måling
Prækondition	Affugteren kører, og den seneste måling er ældre end sensorens gyldighedsperiode (30 minutter for Zigbee-sensoren, 3 minutter for DHT11-sensoren)
Postkondition	Affugteren er slukket, fejlen er logget i databasen og vist på websiden, og styringen er genoptaget, når sensoren igen leverer gyldige målinger
Beskrivelse	Uden en gyldig måling kan systemet ikke afgøre, om affugteren skal køre. Systemet slukker derfor affugteren, indtil sensoren igen leverer data.
Normalt forløb	1. Systemet henter den seneste måling og konstaterer, at den er forældet eller mangler 2. Systemet slukker smart plug'en og gemmer sluk-hændelsen med begrundelse 3. Fejlen logges i databasen og vises på websiden 4. Ved hver efterfølgende styringscyklus forsøger systemet igen at hente en gyldig måling 5. Sensoren leverer igen en gyldig måling, og de almindelige regler gælder

	(se 3.0)
Alternativt forløb	1.1 Affugteren er allerede slukket: den forbliver slukket, og ingen sluk-hændelse gemmes 5.1 Minimumsintervallet er ikke gået: affugteren tændes først, når de 15 minutter er gået
Udvidelsesmuligheder	
Relaterede krav	K-001, K-009, K-012, K-016

Use Case	Strømafbrydelse og genstart
ID	8.0
Aktør(er)	Systemet (primær), systemd, smart plug, elpris-API
Trigger	Strømmen til Raspberry Pi'en afbrydes og tilsluttes igen
Prækondition	Affugteren kører, og smart plug'en er indstillet til at forblive slukket efter en strømafbrydelse
Postkondition	Systemet kører igen og træffer beslutninger hvert 15. minut, uden at brugeren har foretaget sig noget
Beskrivelse	Efter en strømafbrydelse starter systemet automatisk op igen og genoptager styringen. Affugteren tændes først, når minimumsintervallet er gået, så kompressoren beskyttes.
Normalt forløb	1. Strømmen afbrydes, og både Raspberry Pi'en og affugteren slukkes 2. Strømmen vender tilbage, Raspberry Pi'en starter, og systemd starter applikationen (se 1.0) 3. Systemet kontrollerer smart plug'en, henter elpriser og beregner prisgrænsen 4. Styringsløggen starter, og affugteren tændes tidligst, når minimumsintervallet er gået 5. Systemet kører videre efter de almindelige regler (se 3.0)
Alternativt forløb	2.1 Netværket er endnu ikke klar: elpriserne kan ikke hentes, og systemet starter i sikkerhedstilstand (se 6.0) 4.1 Programmet går i stå under drift: systemd genstarter det automatisk, og forløbet gentages fra trin 3
Udvidelsesmuligheder	
Relaterede krav	K-002, K-003, K-006, K-007, K-014, K-015

Use Case	Brugeren følger systemets tilstand
ID	9.0
Aktør(er)	Bruger (primær), systemet
Trigger	Brugeren åbner websiden i en browser
Prækondition	Brugerens enhed er på det samme lokale netværk som Raspberry Pi'en, og systemet kører
Postkondition	Brugeren kender systemets aktuelle tilstand og eventuelle fejl
Beskrivelse	Brugeren kan følge luftfugtigheden, affugterens tilstand og de seneste målinger på en lokal webside uden adgang til databasen.
Normalt forløb	1. Brugeren åbner websiden på Raspberry Pi'ens adresse 2. Siden viser aktuel luftfugtighed og temperatur med tidspunkt og kilde 3. Siden viser, om affugteren er tændt eller slukket, og den aktuelle driftstilstand 4. Siden viser dagens prisgrænse og den aktuelle elpris 5. Brugeren ser i historikken, hvornår affugteren har kørt 6. Siden viser aktive fejl, hvis der er nogen
Alternativt forløb	1.1 Brugeren er ikke på det lokale netværk: siden kan ikke åbnes 6.1 Der vises en sensorfejl: brugeren kontrollerer sensoren, fx batteriet (se 7.0)
Udvidelsesmuligheder	At brugeren kan ændre grænseværdier og tidsrummet for dark time direkte på websiden Visning af systemets aktuelle driftstilstand (normal, kritisk, dark time, sensorfejl, sikkerhedstilstand) direkte på websiden
Relaterede krav	K-009, K-010, K-016

2.5 Prioriteret regelrækkefølge

Affugterens tilstand afgøres af flere forhold på samme tid: luftfugtighed, elpris, tidspunkt, minimumsinterval og eventuelle fejl. For at sikre, at systemets opførsel er entydig i alle kombinationer af disse forhold, gennemgår systemet reglerne nedenfor i fast rækkefølge hvert 15. minut. Den første regel, hvis betingelse er opfyldt, afgør affugterens tilstand. De efterfølgende regler gennemgås ikke. Regel 8 opfanger alle tilfælde, som ikke er dækket af regel 1–7, så der ikke opstår udefinerede situationer.

Nr.	Betingelse	Handling	Krav
1	Der findes ingen gyldig måling fra nogen sensor	Affugteren slukkes, og fejlen logges	Nyt K-012 forsøg ved næste styringscyklus

2	Dark time (klokken er mellem 22.00 og 06.00)	Affugteren slukkes / forbliver slukket	K-018
3	Affugteren er slukket, og der er gået mindre end 15 minutter siden sidste slukning	Affugteren forbliver slukket	K-006
4	Luftfugtigheden er 45% RH eller derunder	Affugteren slukkes / forbliver slukket	K-004
5	Luftfugtigheden er 70% RH eller derover	Affugteren tændes / forbliver tændt	K-005
6	Der findes ingen gyldige elpriser, heller ikke gemte (sikkerhedstilstand)	Affugteren slukkes / forbliver slukket	K-008, K-011
7	Det aktuelle kvarter er billigt	Affugteren tændes / forbliver tændt	K-004
8	Ingen af ovenstående	Affugteren slukkes / forbliver slukket	-

Sensorfejl (regel 1) står først, fordi ingen af de øvrige regler kan anvendes uden en gyldig måling.

Dark time (regel 2) står før alle regler, der tænder affugteren, fordi brugeren har valgt, at affugteren altid skal være slukket om natten af hensyn til støj.

Minimumsintervallet (regel 3) står før alle regler, der tænder affugteren, så kompressoren altid beskyttes.

Slukgrænse (regel 4) står først af fugtreglerne, så affugteren altid slukkes, når luftfugtigheden er nede på 45% RH.

Den kritiske grænse (regel 5) står før elprisreglerne (regel 6-7), så luftfugtigheden prioriteres over elprisen uden for dark time.

Da regel 5 gennemgås før regel 6, kan affugteren i sikkerhedstilstand fortsat tændes ved kritisk grænsen og køre til slukgrænsen, selvom der ikke findes elpriser.

Falder luftfugtigheden under den kritiske grænse, gælder regel 7 igen. Er kvarteret ikke billigt, slukkes affugteren derfor (regel 8), selvom luftfugtigheden endnu ikke er nede på 45% RH.

Example table:

Situation	Første regel, der passer	Resultat
Kl 14.00, ingen gyldig måling	1	Slukket
Kl. 23.00, 75% RH	2	Slukket (dark time)
Kl. 14.00, 75% RH, slukket for 5 minutter før	3	Slukket (venter på minimumsintervallet)
Kl. 14.00, 44% RH	4	Slukket
Kl. 14.00, 75% RH, dyrt kvarter	5	Tændt

Situation	Første regel, der passer	Resultat
Kl. 14.00, 66% RH, tændt, dyrt kvarter	8	Slukket (kritisk drift afbrydes af prisen)
Kl. 14.00, 60% RH, ingen pris overhovedet	6	Slukket
Kl. 14.00, 50% RH, billigt kvarter	7	Tændt
Kl. 14.00, 50% RH, dyrt kvarter	8	Slukket

2.6.1 Ikke-funktionelle krav

Afsnit	Krav-ID	Krav	Kilde
Performance	K-001	Systemet træffer en beslutning hvert 15. minut (styringscyklus). Se K-001 i afsnit 2.3.2 for målingernes gyldighed.	Gruppen
Sikkerhed	-	Systemet anvendes udelukkende på det lokale netværk.	Afgrænsning
Tilgængelighed	K-011	Hvis der ikke findes gyldige elpriser for hvert af døgnets 96 kvarterer, heller ikke gemte, går systemet i sikkerhedstilstand. I så fald tændes affugteren kun, når luftfugtigheden når den kritiske grænse, og kører, indtil den er under slukgrænsen. Når gyldige elpriser igen kan hentes, forlader systemet sikkerhedstilstand, og de almindelige styringsregler gælder.	
	K-012	Systemet skal anvende den seneste gyldige måling (se K-001). Findes der ingen gyldig måling fra nogen sensor, skal affugteren slukkes, fejlen logges i databasen, og systemet skal forsøge igen ved næste styringscyklus (15 minutter).	
	K-013	Hvis smart plug'en ikke svarer, skal fejlen logges i databasen og vises på websiden (K-016), og systemet skal forsøge igen ved næste styringscyklus (15 minutter). Måling og lagring af luftfugtighed og elpriser fortsætter.	
	K-014	Efter en strømafbrydelse skal systemet starte automatisk, hente elpriser, beregne dagens prisgrænse og genoptage styringen uden brugerens hjælp. Affugteren må først tændes, når minimumsintervallet er gået (se K-006).	
	K-015	Programmet skal køre som en systemtjeneste (systemd), der genstarter programmet automatisk, hvis det stopper, og starter det ved opstart af Raspberry Pi'en.	
	K-017	Systemet skal kunne køre uafbrudt i mindst [72] timer uden menneskelig indgriben.	

Brugervenlighed	K-016	Se K-016 i afsnit 2.3.2 (lokal webside med systemets tilstand).	Brugeren
Skalerbarhed	-	Løsningen omfatter styring af én affugter.	Afgrænsning
Lovkrav	-	Der stilles ingen særlige lovkrav ud over gældende regler for elektrisk udstyr.	Gruppen

2.6.2 Datakrav

Krav-ID	Hvad gemmes?	Hvorfor?	Hvor længe?
K-008	Dagens kvarterpriser	Styringen kan fortsætte ved kortvarigt internetafbrydelse	48 timer
K-009	Tidspunkt for hver tænding og slukning samt begrundelse	Dokumenterer, at K-004, K-005, K-006 og K-018 overholdes	Fastlægges af gruppen
K-010	Hver luftfugtighedsmåling med tidspunkt og kilde	Viser udviklingen og bruges til visualisering og test	Fastlægges af gruppen

Alle data gemmes i en PostgreSQL-database på Raspberry Pi'en.

2.6.3 Integrationer og afhængigheder

Krav-ID	Afhængighed	Protokol	Ved fejl
K-001	Zigbee-luftfugtighedssensor, batteridrevet (anvendes hvis ENABLE_ZIGBEE=true)	Zigbee via USB-dongle og Zigbee2MQTT	Se K-012
K-001	DHT11-sensor på GPIO D4 (anvendes hvis ENABLE_DHT11=true)	Intern komponent på Raspberry Pi'en	Se K-012
K-007	Shelly Plug S Gen3, styres lokalt uden cloud	HTTP	Se K-013
K-003	Ekstern elpris-API (fx Energi Data Service), prisområde DK1	HTTPS	Se K-011
-	Affugteren (eese Emil 2508)	Styres kun via strømmen	Forudsætning, se afsnit 1.3

2.7 Prioritering

Kravene er prioriteret efter MoSCoW-metoden. Must er krav, som løsningen ikke kan fungere uden. 'Should' er krav, som gør løsningen mere robust, men ikke er nødvendige for grundfunktionen. 'Could' er krav, som er nyttige, men kan udelades.



Prioritet	Krav-ID	Begrundelse
Must	K-001, K-002, K-003, K-004, K-005, K-006, K-007, K-018	Kernen i styringen: måling, elpris, beslutning og styring af affugteren.
Must	K-009	Tænd/sluk loggen bruges til at dokumentere, at styringen virker
Must	K-011, K-012, K-014, K-017	Opgavebeskrivelsen kræver fejlhåndtering og drift over længere tid
Must	K-016	Opgavebeskrivelsen kræver, at systemets tilstand kan følges
Should	K-008, K-013, K-015	Gør systemet mere robust, men grundfunktionen virker uden
Could	K-010	Nyttig historik, men ikke nødvendig for styringen

2.7.1 Sikkerhedstilstand og failure modes

Sikkerhedstilstanden er udviklet for at sikre fortsat drift på systemet i et scenarie, hvor elpriser ikke kan indhentes. Herunder to scenarier:

- Både Raspberry Pi'en og smart plug'en er på et fælles netværk, men netværket har ikke internetforbindelse;
- Fejl på API-siden, så elprisdata ikke er tilgængelige.

Hvis elprisdata tidligere er indhentet, vil systemet fortsætte med at matche med den gemte pris for det aktuelle kvarter.

Hvis, derimod, det ikke er lykkedes at indhente priser tidligere, og API'et fortsat er utilgængeligt, går systemet i sikkerhedstilstand. I det tilfælde kører systemet udelukkende på

sensordata, og vil kun tænde for affugteren, når den målte luftfugtighed overstiger den kritiske grænse på 70% RH eller derover.

Ligeledes vil affugteren først slukkes, når luftfugtigheden igen er 45% RH eller derunder.

Mens systemet er i sikkerhedstilstand, vil det fortsat løbende, hvert kvarter, forsøge at indhente priser fra API'et. Sikkerhedstilstanden fortsætter, indtil nye prisdata kan indhentes. I "dark time" er affugteren slukket, uanset om systemet er i sikkerhedstilstand (K-018).

Andre failure modes:

Systemet er som udgangspunkt i stand til at køre uden input og forbindelse – dog med visse undtagelser.

Overordnet set består systemet af en applikation på Raspberry Pi'en, samt fem mere eller mindre eksterne komponenter:

-Smart plug

Er det ikke muligt at oprette forbindelse til smart plug'en, vil systemet køre videre, og forsøge at genetablere forbindelsen hvert 15. minut.

Applikationen kan fint køre, men systemet anses for ikke at være funktionelt uden forbindelse til en smart plug. Fejl logges i databasen.

- Elprisdata-API

Uden API-data går systemet i sikkerhedstilstand, og fortsætter udelukkende med sensordata.

Applikationen er i princippet funktionel uden, men må forsøges udbedret ved fortsat drift. Fejl logges i databasen.

- Luftfugtighedssensor

Hvis der ikke kan indhentes data fra sensoren, kan elprisdata ikke anvendes, da styringen kræver en gyldig måling. Affugter er slukket eller forbliver slukket. Systemet vil fortsat køre, og forsøge at indhente data fra sensoren med 5 retries hvert 15. minut ved næste styringscyklus. Fejl logges i databasen.

- Internetforbindelse

Som beskrevet foroven er manglende internetforbindelse svarende til manglende adgang til elprisdata, forudsat at både smart plug og Raspberry Pi er på samme netværk. Systemet fortsætter ligeledes i sikkerhedstilstand, indtil forbindelse kan genoprettes, og forsøger at genetablere kontakt med API'et. Fejl logges i databasen.

- Database

Systemet anses for ikke at være funktionelt uden databasen. Databasen ligger lokalt på Raspberry Pi'en, og er essentiel for almindelig drift af systemet.

3. Brugervejledning

Systemet kan opsættes med en DHT11-sensor (variant A, kablet) eller en Zigbee-sensor (variant B, trådløs). Trin, der kun gælder den ene variant, er markeret med A eller B.

3.1 Hardware-opsætning

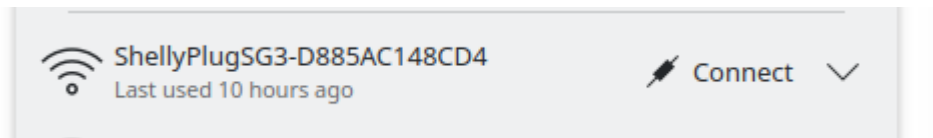
Komponent	Model	Variant
Raspberry Pi med strømforsyning og microSD kort	Raspberry Pi 4 / 5, Raspberry Pi OS (64-bit)	A + B
Smart plug	Shelly Plug S Gen3	A + B
Affugter	eeese Emil 2508	A + B
Sensor	DHT11-modul (3 ben) + 3 jumper kabler	A
Sensor og USB-dongle	Zigbee-sensor + Sonoff ZBDongle-E (adapter:ember) med USB forlængerkabel	B

Raspberry Pi

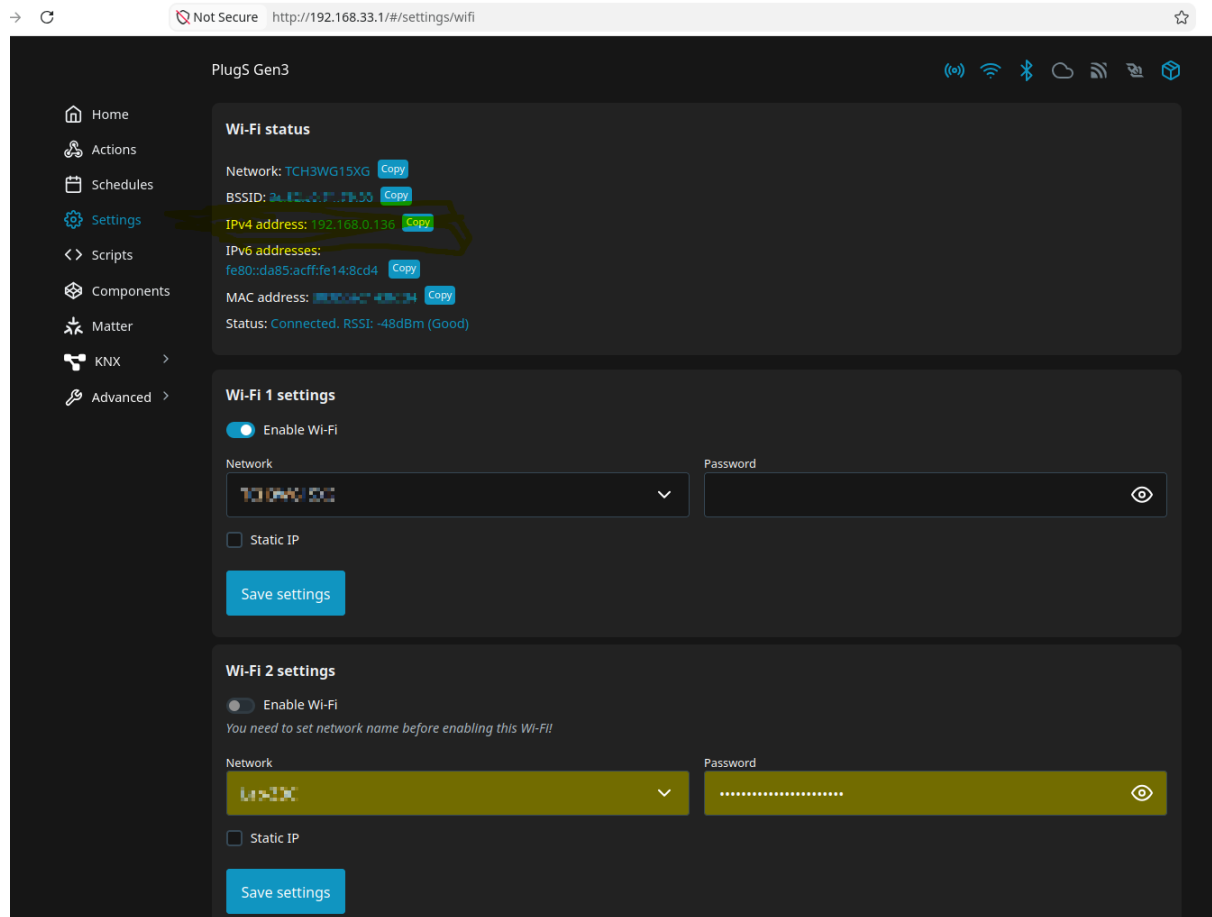
Installer Raspberry Pi OS med Raspberry Pi Imager, og aktiver SSH. Log ind med `ssh <bruger>@<pi-navn>.local`, og sæt tidszonen: `sudo timedatectl set-timezone Europe/Copenhagen`.

Shelly og affugter

1. Sæt Shelly-stikket i stikkontakten, forbind til dets Wi-Fi (ShellyPlugSG3-...), og åbn <http://192.168.33.1>.



2. Tilslut stikket til samme netværk som Pi'en under **Settings** -> **Wi-Fi**.



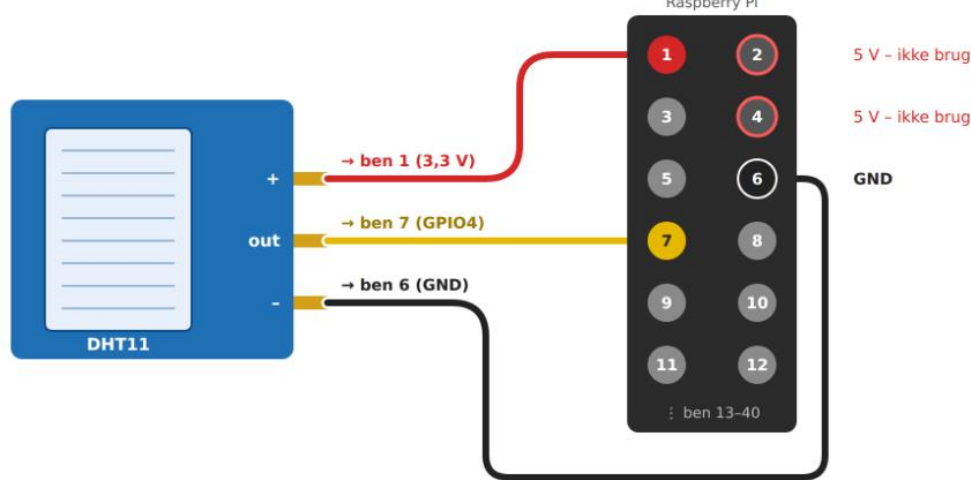
3. Nu får Shelly en IP-adresse, find stikkets id med: `curl -s http://<shelly-ip>/shelly`. Værtsnavnet er `<id>.local`
4. Sæt affugteren i Shelly-stikket, og tænd den på dens egen knap, så den genoptager driften, når strømmen kommer igen.

A: DHT11

DHT11	Raspberry Pi
+ (VCC)	ben 1 (3,3 V)
out (DATA)	ben 7 (GPIO4)
- (GND)	ben 6

Tilslutning af DHT11 til Raspberry Pi

Set ovenfra · GPIO-header, ben 1-12 (ben 13-40 bruges ikke)



Ben 1 har en firkantet loddeø. De ulige ben (1, 3, 5 ...) sidder i den inderste række.
Sluk Raspberry Pi'en før tilslutning, og brug 3,3 V (ben 1) – ikke 5 V (ben 2 og 4).

B: Zigbee

Sæt donglen i en USB-port via et forlængerkabel, så donglen sidder et stykke fra pi'en og dens blå USB 3-porte, som kan forstyrre radiosignalet.

Find dens sti med: `ls -l /dev/serial/by-id`. Forventet resultat: en linje, der begynder med usb-SONOFF...

Sæt batteri i sensoren. Sensoren parres i afsnit 3.2.

3.2 Installation

3.2.1 Systempakker

```
sudo apt update && sudo apt install -y git python3-venv postgresql
```

Variant A (DTH 11) kræver desuden værktøjer til at læse GPIO-benene:

```
sudo apt install -y python3-dev build-essential libgpiod3 swig liblpgpio-dev
```

Variant B (Zigbee): Mosquitto, Zigbee2MQTT og parring

```
sudo apt install -y mosquitto mosquitto-clients
```

Installér **Zigbee2MQTT** efter den officielle Linux-vejledning (zigbee2mqtt.io -> Installation -> Linux), inklusive afsnittet om at køre den som systemd-tjeneste, så den starter automatisk ved opstart. Sæt i dens **configuration.yaml**:

```
mqtt:
```

server: mqtt://localhost:1883

serial:

port: /dev/serial/by-id/<donglens navn>

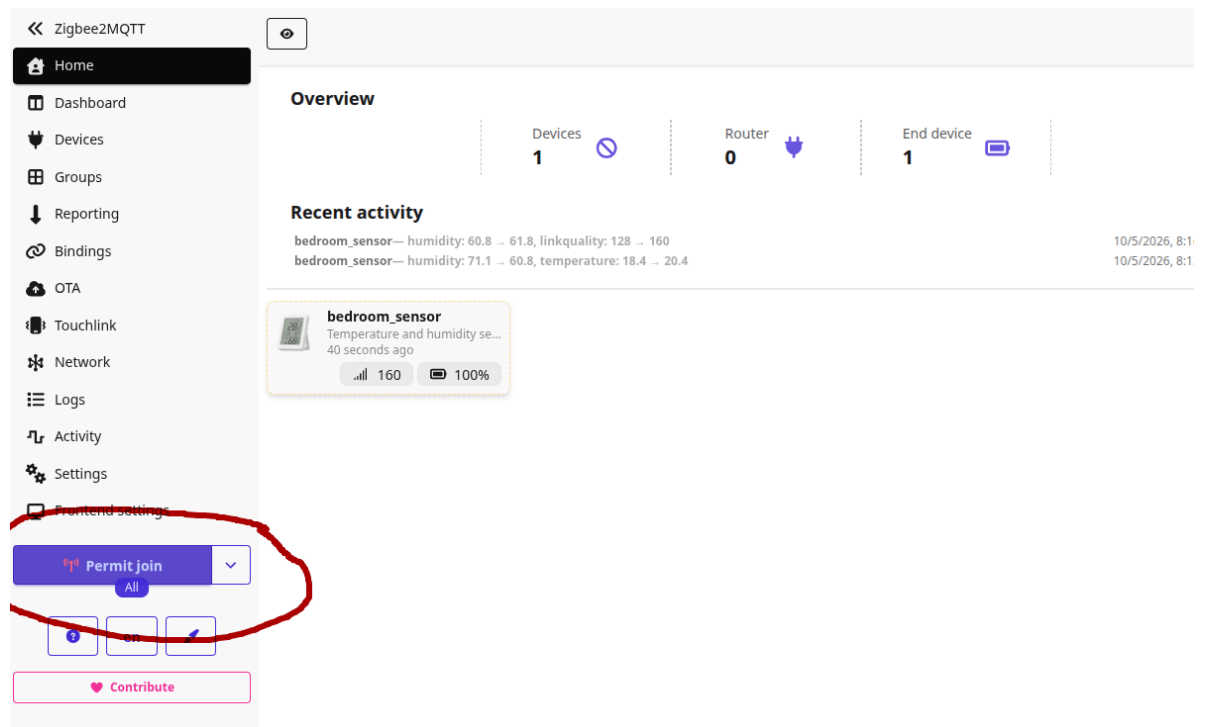
adapter: ember

frontend:

port: 8080

Parring af sensoren:

1. Åbn Zigbee2MQTT's webside i en browser: på en computer på samme netværk via `http://<pi-ip>:8080`, eller direkte på Pi'en via <http://localhost:8080>, hvis den har skrivebord. Klik på **Permit join**.



2. Hold sensorens knap inde i ca. 5 sekunder, til den blinker.
3. Omdøb sensoren under Devices til et navn uden /, fx `bedroom_sensor`.

3.2.2 Hent kildekoden og installer Python-pakker

```
cd ~
```

```
git clone -b zigbee-integration https://github.com/Tian70777/affugter.git
```

```
cd ~/affugter
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install -r requirements.txt
```

```
pip install adafruit-circuitpython-dht # kun variant A
```

3.2.3 Database

Opret en database bruger, en database og tabellerne. De to sidste kommandoer beder om adgangskoden fra første linje.

```
sudo -u postgres psql -c "CREATE USER affugter WITH PASSWORD  
'<adgangskode>';"
```

```
sudo -u postgres psql -c "CREATE DATABASE humidifier OWNER affugter;"
```

```
psql -h localhost -U affugter -d humidifier -f database/init.sql
```

```
psql -h localhost -U affugter -d humidifier -c '\dt'
```

Forventet resultat: fire tabeller: electricity_prices, errors, humidifier_state og humidity.

3.2.4 Konfigurationsfilen .env

Opret filen ud fra skabelonen med `cp .env.example .env`, og rediger den med `nano .env`:

Variabel	Variant A: DTH11	Variant B: Zigbee	Beskrivelse
POSTGRES_HOST	localhost	localhost	databasens adresse
POSTGRES_PORT	5432	5432	databasens port
POSTGRES_DB	humidifier	humidifier	databasens navn
POSTGRES_USER	affugter	affugter	database bruger
POSTGRES_PASSWORD	<adgangskode>	<adgangskode>	adgangskoden fra 3.2.3
SHELLY_HOST	<shelly-id>.local	<shelly-id>.local	smart plug'ens værtsnavn (3.1.3)
MQTT_HOST	bruges ikke	localhost	MQTT-brokerens adresse
ENABLE_ZIGBEE	false	true	start Zigbee modtageren
ENABLE_DHT11	true	false	Start DHT11-aflæsningen

3.2.5 Start som tjeneste

Opret `/etc/systemd/system/affugter.service`:

[Unit]

Description=Affugter

After=network-online.target postgresql.service

[Service]

User=<bruger>

WorkingDirectory=/home/<bruger>/affugter

Environment=PYTHONUNBUFFERED=1

ExecStart=/home/<bruger>/affugter/venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000

Restart=always

[Install]

WantedBy=multi-user.target

sudo systemctl daemon-reload

sudo systemctl enable --now affugter

Programmet starter nu automatisk ved opstart (K-015). Dashboardet åbnes på `http://<pi-ip>:8000/dashboard/`, og loggen følges med `journalctl -u affugter -f`.

Kør `systemctl is-enabled postgresql affugter` (variant B: `systemctl is-enabled postgresql mosquitto zigbee2mqtt affugter`).

Forventet resultat: enabled for hver tjeneste.

3.3 Anvendelse af websiden

Til applikationen hører to lokalt hostede websider:

Dashboard med overblik over aktuelle og historiske elpriser og sensordata, samt en aktivitetslog og error log.

En debug-orienteret side med manuelle kontroller, f.eks. tænd/sluk og testværdier.

3.3.1 Dashboard

Opret filen `~/affugter/.env` ud fra skabelonen, og tilpas værdierne:

```
cp .env.example .env
```

```
nano .env
```



Dashboardet er ikke udviklet til at være interaktivt. Det er udelukkende en side, som giver brugeren overblik over aktuelle og historiske data.

Luftfugtighed

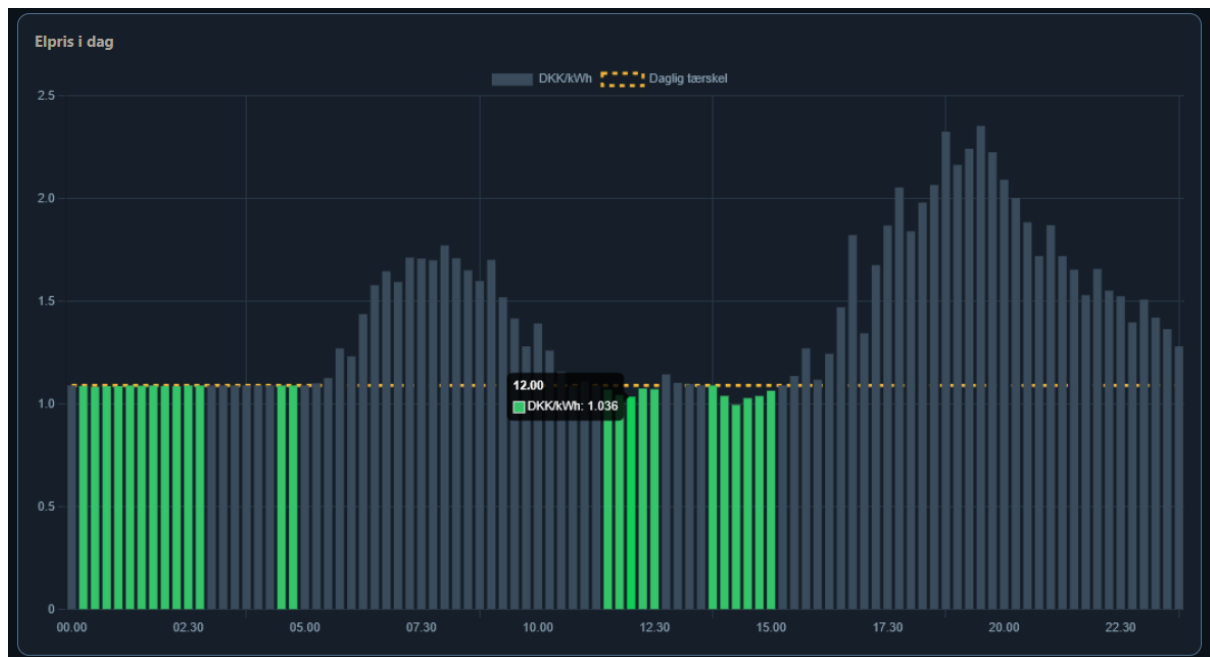
Under 'Luftfugtighed' vises der en graf med luftfugtighedsmålinger med timestamps over de sidste 24 timer fra siden åbnes. Grafen er farvekodet, så brugeren let kan tyde hvor længe luftfugtigheden har været over eller under de prædefinerede værdier.

Temperatur

Temperatur er ikke et vigtigt datapunkt for projektet, men opsamles automatisk gennem systemets DHT11-sensor, og vises ved siden af luftfugtighedsdata for at give et bredere billede af opsamlede data.

Elpris i dag

Elprisoversigten viser indhentede elprisdata for det aktuelle døgn, hvor siden åbnes. Man kan med musen fremhæve prisen ned til det enkelte kvarter på døgnet:



Highlightet i grøn vises alle de kvarterpriser, som ligger under den bestemte pristærskel på 25% - det vil sige døgnet 25% laveste priser.

3.3.2 Aktivitetslog

Seneste ændringer af affugter		
Tid	Tilstand	Årsag
1.10.2026, 09.15.00	SLUKKET	Humidity 69.1% within range, but electricity price 1.5198 DKK/kWh is above threshold 1.0908 DKK/kWh
1.10.2026, 06.00.00	TÆNDT	Emergency threshold crossed: humidity 74.8% >= 70%
30.9.2026, 15.30.00	SLUKKET	Humidity 52.8% within range, but electricity price 0.7859 DKK/kWh is above threshold 0.718 DKK/kWh
30.9.2026, 11.30.00	TÆNDT	Humidity 68.5% is within range, and electricity price 0.5439 DKK/kWh is below threshold 0.718 DKK/kWh
30.9.2026, 11.00.00	SLUKKET	Humidity 68.5% within range, but electricity price 0.8152 DKK/kWh is above threshold 0.718 DKK/kWh
30.9.2026, 10.45.00	TÆNDT	Humidity 68.5% is within range, and electricity price 0.6503 DKK/kWh is below threshold 0.718 DKK/kWh
30.9.2026, 09.45.00	SLUKKET	Humidity 69.1% within range, but electricity price 1.012 DKK/kWh is above threshold 0.718 DKK/kWh
30.9.2026, 06.00.00	TÆNDT	Emergency threshold crossed: humidity 76.7% >= 70%
29.9.2026, 22.00.00	SLUKKET	Dark time active: current time 22:00 is between 22:00 and 06:00
29.9.2026, 20.45.00	TÆNDT	Emergency threshold crossed: humidity 70.3% >= 70%
29.9.2026, 16.15.00	SLUKKET	Humidity 60.3% within range, but electricity price 1.0916 DKK/kWh is above threshold 1.0378 DKK/kWh

Hver gang stikproppen tændes og slukkes, bliver det logget i databasen. Aktivitetsloggen viser hvilken handling der er foretaget, hvornår, og hvilken begrundelse programmet har tilknyttet.

3.3.3 Error log

Seneste systemfejl		
Tid	Type	Fejl
1.10.2026, 10.49.57	RuntimeError	A full buffer was not returned. Try again.
1.10.2026, 10.43.23	RuntimeError	A full buffer was not returned. Try again.
1.10.2026, 10.37.50	RuntimeError	Checksum did not validate. Try again.
1.10.2026, 10.33.47	RuntimeError	A full buffer was not returned. Try again.
1.10.2026, 10.29.45	RuntimeError	Checksum did not validate. Try again.
1.10.2026, 10.27.13	RuntimeError	Checksum did not validate. Try again.
1.10.2026, 10.25.12	RuntimeError	A full buffer was not returned. Try again.
1.10.2026, 10.21.09	RuntimeError	A full buffer was not returned. Try again.
1.10.2026, 10.18.38	RuntimeError	A full buffer was not returned. Try again.
1.10.2026, 10.16.06	RuntimeError	A full buffer was not returned. Try again.

Error loggen viser alle opsamlede errors i applikationen. Hver gang en fejl håndteres, gemmes den i databasen, og vises på sidens error log.

3.3.4 Debug

Humidifier Control

Debug and manual control interface

Fixed tests

High price

Low price

High humidity

Low humidity

Real values

Shelly

Turn ON

Turn OFF

Server-based control

Start

Stop

Humidity

Relative humidity (%)

e.g. 65

Test

Electricity price

DKK / kWh

0.5

Test

Custom test

Humidity (%)

e.g. 60

Electricity price (DKK / kWh)

e.g. 0.50

Run test

Response

No request sent yet.

Idle

Debug-panelet er bygget til test, og har til formål at sende systemet specialiserede inputs.

Det er f.eks. muligt manuelt at tænde og slukke for stikproppen. Det er ligeledes muligt at sende de særlige luftfugtighedsværdier og elpriser, for at verificere, at systemet opfører sig efter hensigten.

Forklaring af de enkelte muligheder under Fixed tests:

High price: Sender en falsk høj energipris: 100 kr./kWh

Low price: Sender en falsk lav energipris: 10 øre/kWh

High humidity: Sender en falsk høj luftfugtighedsmåling: 100%

Low humidity: Sender en falsk lav luftfugtighedsmåling: 0%

Real values: Nulstiller systemet til at bruge aktuelle, læste og gemte sensordata og energipriser.

Shelly: Tillader manuel tænd og sluk af stikproppen.

Server-based control: Tillader manuel start og stop af systemets drift som helhed.

Humidity: Sender en specifik, indtastet luftfugtighedsmåling. Har til formål at teste, at systemet fungerer til hensigten ved særlige værdier.

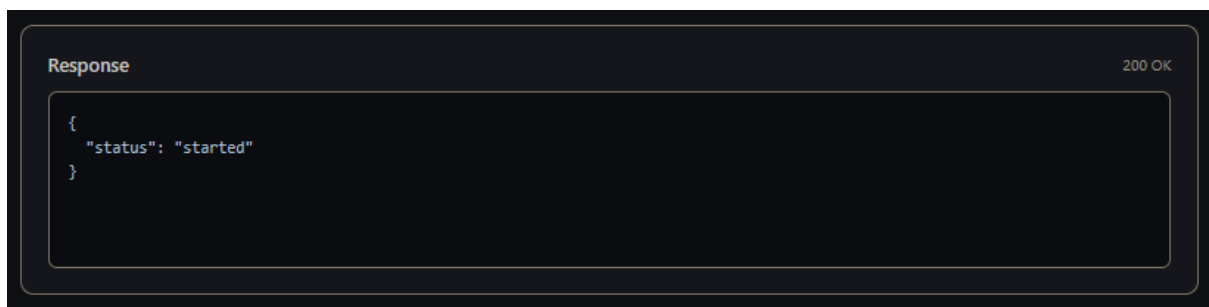
Electricity price: Sender en specifik, indtastet energipris. Har til formål at teste, at systemet fungerer til hensigten ved særlige værdier.

Custom test: Sender en specifik kombination af luftfugtighed og energipris.

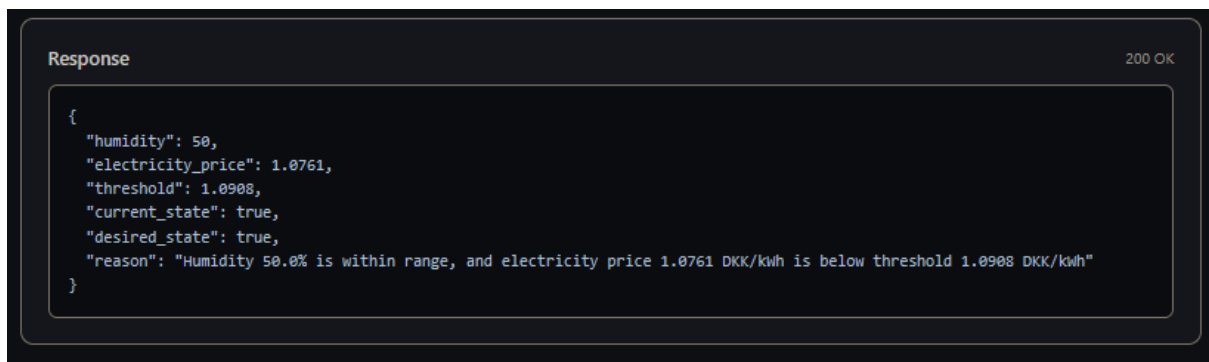
Response

Viser systemets respons på hver af de ovenstående API-kald, f.eks.

Server-based control -> Start:



Eller Fixed tests -> Real values:



4. Teknisk dokumentation

4.1 Arkitektur (API-oversigt)

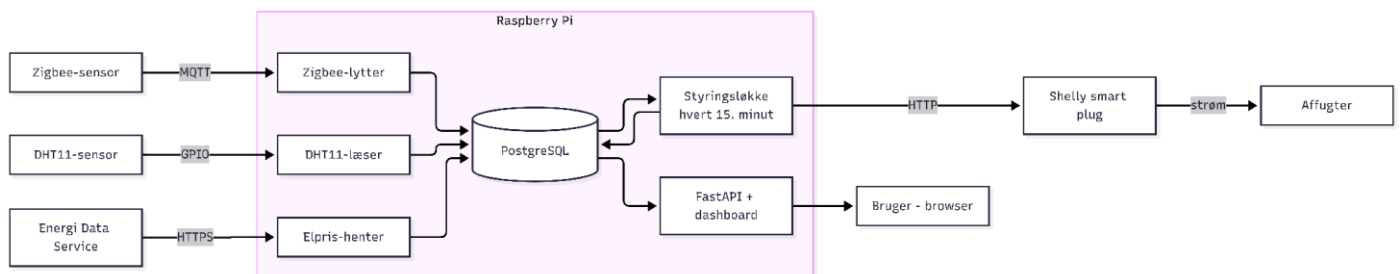
Systemet består af en Raspberry Pi, som kører hele løsningen lokalt. På Raspberry Pi'en kører en FastAPI-applikation og en PostgreSQL-database.

Applikationen har to hoveddele:

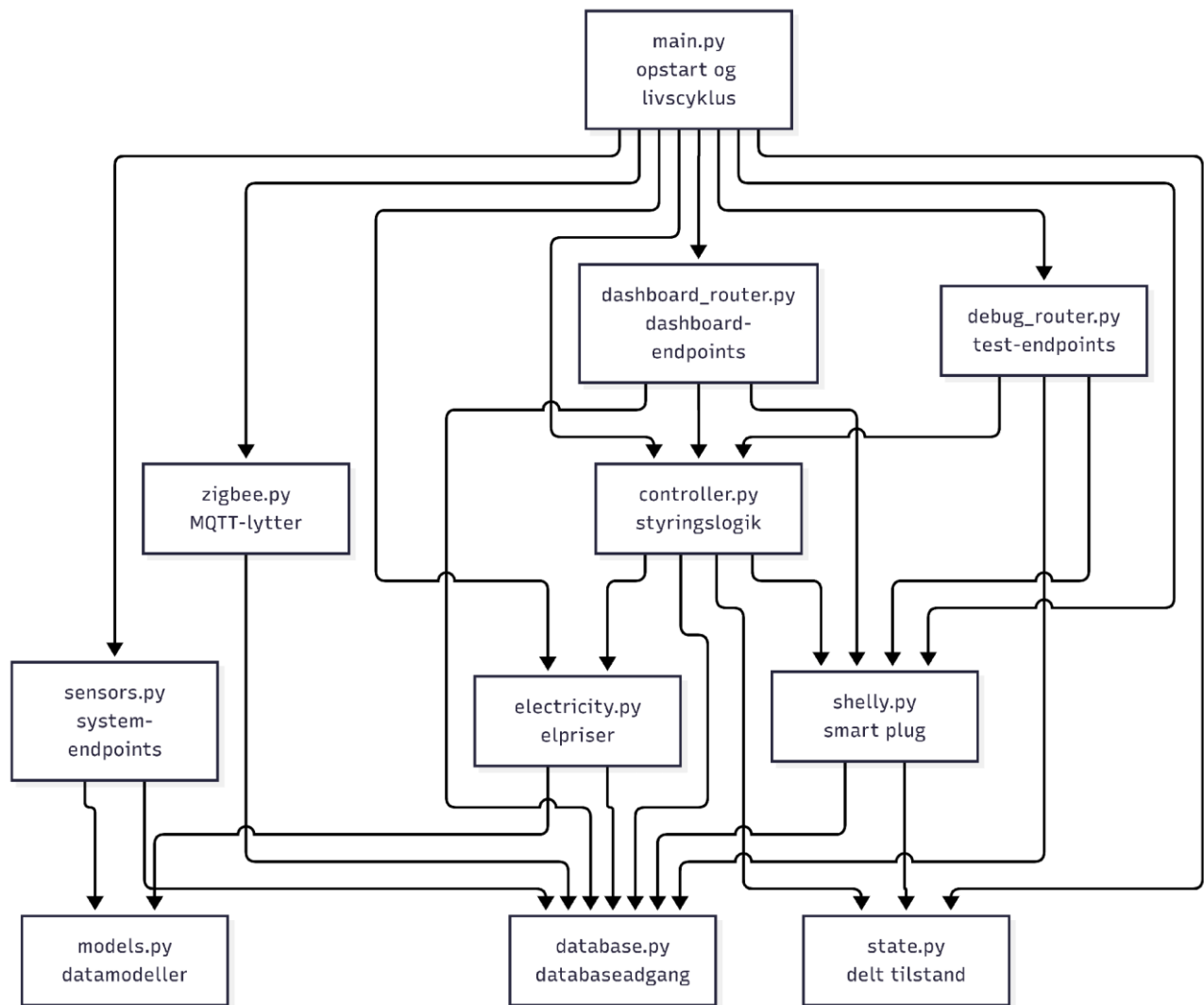
- Dataindsamling: Luftfugtighedsmålinger fra den konfigurerede sensor og kvarterpriser fra Energi Data Service gemmes i databasen.
- Styring: En styringsløkke træffer hvert 15. minut en beslutning ud fra den seneste gyldige måling, dagens prisgrænse og reglerne i afsnit 2.5, og tænder eller slukker smart plug'en via HTTP på det lokale netværk.

Systemets tilstand vises på et lokalt dashboard (K-016).

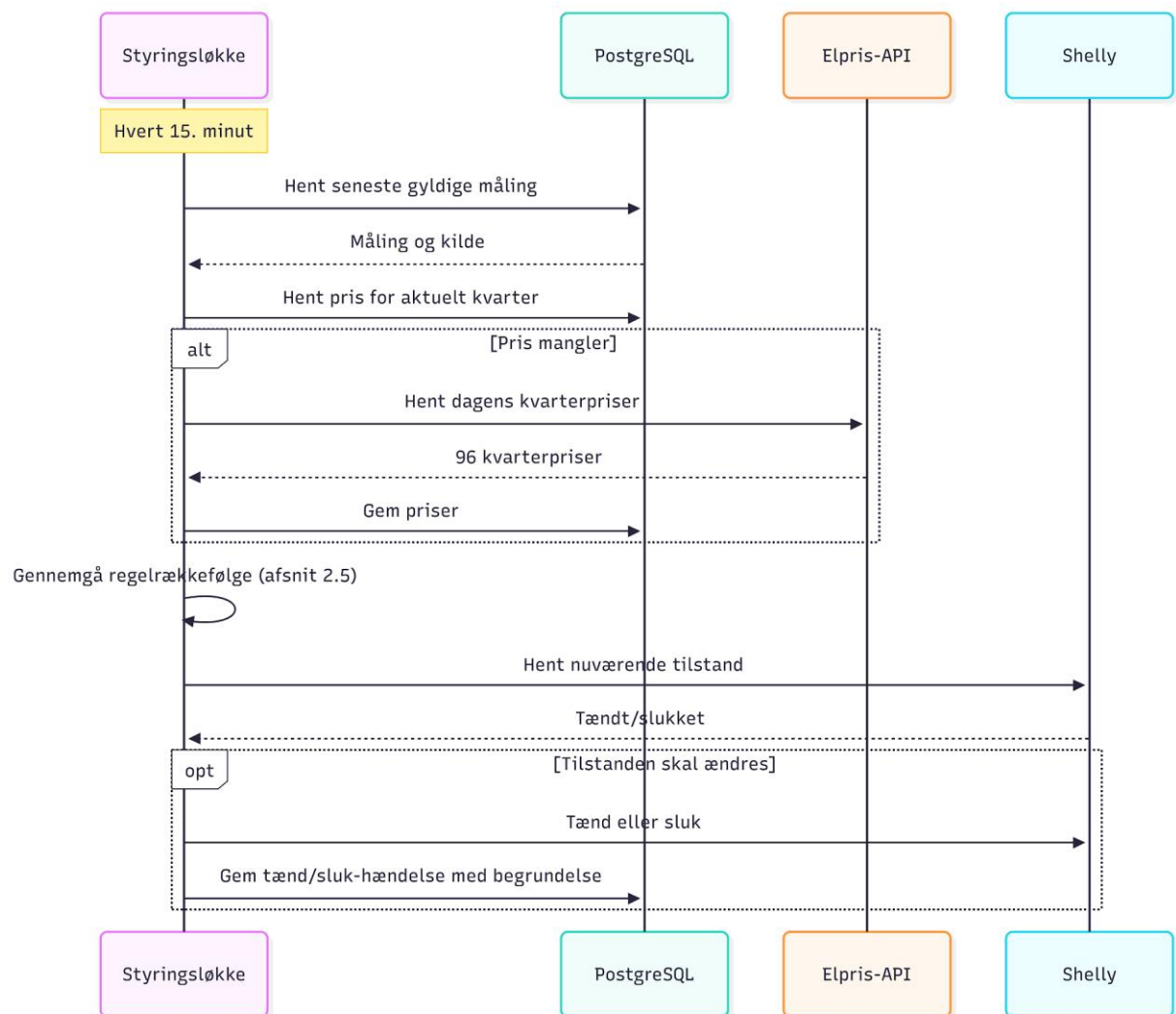
Figur nedenunder viser systemets komponenter og dataflow.



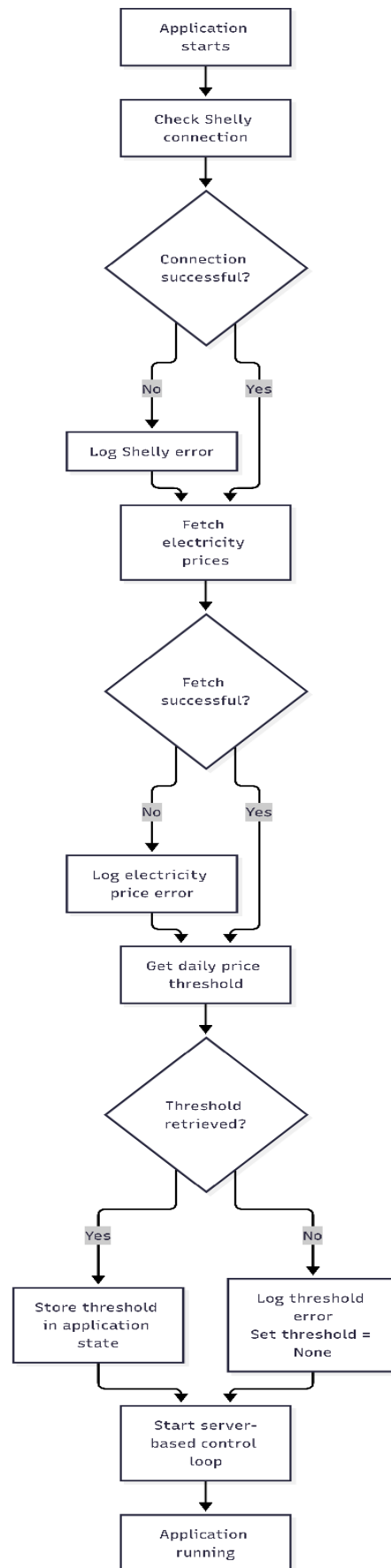
Module- og afhængighedsdiagram:



4.2 Procesdiagrammer (opstart, styringsløkke, determine-state)

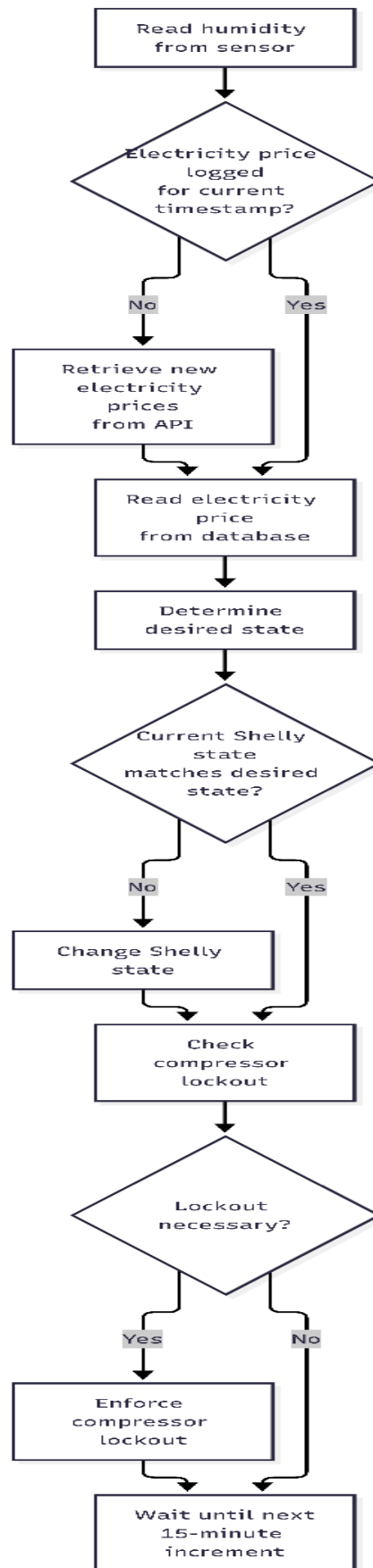


Startup sequence diagram



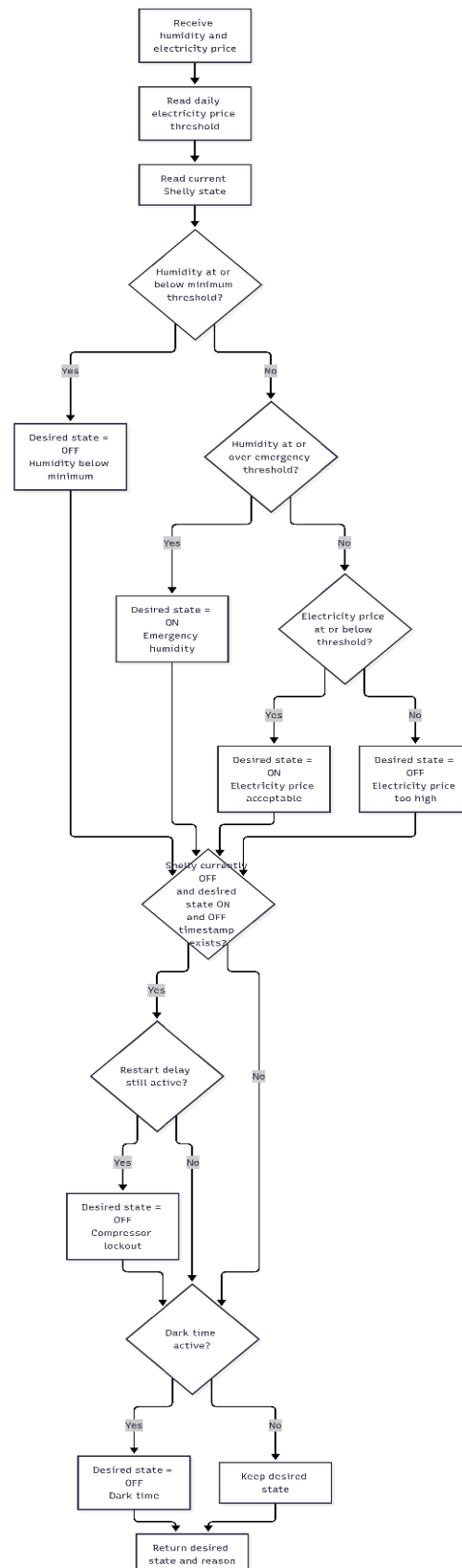
server-based-loop-
minute cycle

process.png — the 15-



determine-state-process.png —

the decision itself



4.3 API-beskrivelse

Figur i bilag 3 viser systemets endpoints, opdelt i tre grupper: system-endpoints til modtagelse af data, dashboard-endpoints til visning og debug-endpoints til test og fejlfinding.

4.3.1 Systemendpoints

Systemendpointsene gør det muligt for en ekstern enhed at sende måledata og tilstandsændringer til systemet over HTTP. De anvendes ikke i den aktuelle opsætning, hvor sensoren er tilsluttet Raspberry Pi'en direkte (DHT11 via GPIO eller Zigbee via USB-dongle), og hvor applikationen skriver målingerne direkte i databasen. Endpointsene er bevaret, så løsningen kan udvides til en opsætning, hvor dataopsamlingen foregår på en separat enhed, og de anvendes desuden til test af dataindgangen.

Endpoint	Metode	Request body	Beskrivelse
/humidity	POST	HumidityReading	Modtager en luftfugtighedsmåling og gemmer den i databasen
/electricity-price	POST	ElectricityPriceReading	Modtager en kvarterpris og gemmer den i databasen
/humidifier-state	POST	ShellyState	Registrerer en tænd/sluk-hændelse med begrundelse

4.3.2 Dashboard-endpoints

Dashboard-endpoints anvendes af websiden til at vise systemets tilstand. Siden henter data fra de fem api-endpoints, når den indlæses og derefter løbende.

Endpoint	Metode	Parametre	Beskrivelse
/dashboard/	GET	-	Viser dashboardwebsiden
/dashboard/api/status	GET	-	Aktuel luftfugtighed, temperatur, sensorkilde, affugterens tilstand, aktuel elpris og dagens prisgrænse
/dashboard/api/history	GET	hours (standard 24)	Historik over luftfugtigheds- og temperaturmålinger
/dashboard/api/prices	GET	-	Dagens kvarterpriser og prisgrænse
/dashboard/api/states	GET	limit (standard 20)	Seneste ændringer af affugterens tilstand med begrundelse
/dashboard/api/errors	GET	ShellyState limit (standard 10)	Seneste loggede systemfejl

4.3.3 Test- og fejlfindingsendpoints

Ud over de endpoints, der indgår i systemets normale drift, indeholder API'et endpoints udviklet til test og fejlfinding. De gør det muligt manuelt at fremprovokere bestemte tilstande og verificere, at systemet reagerer som beskrevet i afsnit 2.5. De er ikke nødvendige for brugerens normale anvendelse af systemet.

Endpointsene kalder styringslogikken direkte med simulerede værdier og omgår dermed styringscyklussen. Endpointsene /test/on og /test/off omgår desuden selve styringslogikken. De er kun tilgængelige på det lokale netværk og bør deaktiveres, hvis systemet gøres tilgængeligt udefra.

Endpoint	Metode	Beskrivelse
/debug/	GET	Viser debug-siden med knapper til de enkelte test
/debug/test/high_price	POST	Kører en beslutning med den aktuelle måling og en simuleret pris på 100 DKK/kWh
/debug/test/low_price	POST	Kører en beslutning med den aktuelle måling og en simuleret pris på 0,10 DKK/kWh
/debug/test/price/{val}	POST	Kører en beslutning med den aktuelle måling og en valgfri pris
/debug/test/high_humidity	POST	Kører en beslutning med en simuleret luftfugtighed på 99 % RH og den aktuelle pris
/debug/test/low_humidity	POST	Kører en beslutning med en simuleret luftfugtighed på 1 % RH og den aktuelle pris
/debug/test/humidity/{val}	POST	Kører en beslutning med en valgfri luftfugtighed og den aktuelle pris
/debug/test/custom/{hum}/{price}	POST	Kører en beslutning med både valgfri luftfugtighed og valgfri pris
/debug/test/real	POST	Kører en beslutning med de aktuelle, rigtige værdier uden at vente på næste styringscyklus
/debug/test/on	POST	Tænder smart plug'en direkte uden om styringslogikken
/debug/test/off	POST	Slukker smart plug'en direkte uden om styringslogikken
/debug/test/server_based	POST	Starter styringsløkken
/debug/test/server_based/stop	POST	Stopper styringsløkken

4.4 Database

HUMIDITY		
serial	id	PK
numeric	percent	
numeric	temperature	
text	source	
timestampz	timestamp	

ELECTRICITY_PRICES		
serial	id	PK
numeric	DKK_per_kWh	
timestampz	time_start	UK
timestampz	time_end	UK

HUMIDIFIER_STATE		
serial	id	PK
boolean	state	
text	reason	
timestampz	timestamp	

ERRORS		
serial	id	PK
text	error_type	
text	error_message	
timestampz	timestamp	

Databasen er en relationsdatabase (PostgreSQL), men tabellerne har ingen fremmednøgler og dermed ingen indbyrdes relationer. Hver tabel gemmer én type hændelse: en måling, en pris, en tilstandsændring eller en fejl. Der findes ingen én-til-mange relation mellem dem, da hver række er en selvstændig observation med sit eget timestamp.

Denne struktur er valgt, fordi den holder skrivningerne uafhængige af hinanden: en sensorfejl forhindrer ikke, at elpriser gemmes, og omvendt (se K-012 og K-013). Sammenhængen mellem tabellerne skabes ud fra timestamps, når data hentes, f.eks. når styringen henter den seneste gyldige måling og prisen for det aktuelle kvarter.

4.5 Biblioteker og teknologivalg

Systemet benytter sig af en række libraries. Her er en oversigt og beskrivelse af dem.

Python-biblioteker

Bibliotek	Version	Funktion
FastAPI	0.141.1	FastAPI er et webframework, som tillader opsætning af API-routes til at modtage HTTP-kald og sende svar gennem systemet.
Uvicorn	0.53.0	Uvicorn er en webserver, som bruges til at køre og servere FastAPI-applikationen.
Psycopg[binary]	3.3.5	Tillader tilslutning til og kommunikation med en PostgreSQL-database.
python-dotenv	1.2.3	Tillader systemet at indlæse environment variables, som opbevares i en .env-fil. Herunder API-nøgler, password til database o.l.
HTTPX	0.28.1	Tillader HTTP-kald ud af systemet – i dette tilfælde for at indhente elpriser fra et API, som eksponerer dem.
adafruit-blinka	-	Et kompatibilitetsbibliotek, som gør det muligt at benytte CircuitPython-biblioteker på Raspberry Pi og giver disse adgang til bl.a. boardets GPIO-pins.
adafruit-circuitpython-dht	-	Et specifikt bibliotek til DHT-temperatur- og luftfugtighedssensorer, som lader systemet aflæse målinger fra en sensor tilsluttet Raspberry Pi-boardets GPIO-pins.
aiomqtt	2.5.1	Asynkron MQTT-klient, som modtager målinger fra Zigbee2MQTT

paho-mqtt 2.1.0 Underliggende MQTT-bibliotek for aiomqtt

Øvrige tjenester på Raspberry Pi'en

Tjeneste	Funktion
PostgreSQL 18	Lokal database til målinger, elpriser, tænd/sluk-hændelser og fejl
Mosquitto	MQTT-broker, som Zigbee2MQTT publicerer til, og som systemet abonnerer på
Zigbee2MQTT	Oversætter Zigbee-beskeder fra USB-donglen til MQTT-beskeder
systemd	Kører applikationen som en tjeneste og genstarter den automatisk (K-015)

5. Testrapport

5.1 Accepttest

Krav-ID	Testhandling	Forventet resultat	Faktisk resultat	Status
K-001	Systemet kører i 10 minutter.	Databasen indeholder målinger fra de tilsluttede sensorer med tidspunkt og kilde.	a. Målinger fra kilden "zigbee" blev gemt med uregelmæssige mellemrum, da sensoren selv sender ved ændringer. Det længste mellemrum var ca. 13 minutter. Målinger fra kilden "dht11" blev gemt ca. hvert 30 sekunder.	Bestået
K-002	Systemet får 96 kendte testpriser.	Prisgrænsen sættes til den 24. laveste pris, og netop 24 kvarterer er billige.	Databasen indeholdt 96 kvarterpriser for den 6. oktober. Den 24. laveste pris var 1,0843 DKK/kWh, og systemet beregnede samme prisgrænse (vist på dashboardet).	Bestået

Krav-ID	Testhandling	Forventet resultat	Faktisk resultat	Status
			Netop 24 kvarterer lå på eller under grænsen.	
K-003	Systemet startes med internetforbindelse og URL til den eksterne API	Dagens 96 kvarterpriser for DK1 hentes og gemmes.	Ved opstart hentede systemet dagens priser fra Energi Data Service. Databasen indeholdt herefter 96 kvarterpriser for den 6. oktober med start- og sluttidspunkt (COUNT(*) = 96).	Bestået
K-004	Luftfugtigheden hæves over 45% RH, (a) i et billigt kvarter og (b) i et dyrt kvarter.	(a) Smart plug'en tændes senest ved næste styingscyklus. (b) Smart plug'en forbliver slukket.	(a) Den 4. oktober kl. 10.30 var luftfugtigheden 67,7 % RH, og prisen 0,9841 DKK/kWh var under prisgrænsen på 1,0394. Affugteren blev tændt. (b) Kl. 10.00 samme dag var luftfugtigheden 69,1 % RH, men prisen 1,1418 DKK/kWh lå over grænsen, og affugteren blev slukket.	Bestået
K-005	Luftfugtigheden hæves til 70% RH i et dyrt kvarter uden for dark time.	Smart plug'en tændes ved næste styingscyklus, uanset elprisen.	Den 4. oktober kl. 08.15 var luftfugtigheden 71,5 % RH, og elprisen 1,4253 DKK/kWh, langt over prisgrænsen på 1,0394. Affugteren blev alligevel tændt med begrundelsen	Bestået

Krav-ID	Testhandling	Forventet resultat	Faktisk resultat	Status
			"Emergency threshold crossed".	
K-006	Affugteren slukkes, og betingelserne for at tænde opfyldes efter 5 minutter.	Affugteren tændes først, når der er gået 15 minutter. Loggen viser tidspunkterne.	Affugteren blev slukket, hvorefter betingelserne for at tænde blev opfyldt efter ca. 5 minutter. Affugteren forblev slukket og blev først tændt, da der var gået 15 minutter siden slukningen. Forløbet fremgår af tænd/sluk-loggen.	Bestået
K-007	Raspberry Pi'en sender tænd- og slukkommandoer, mens routerens internetforbindelse er afbrudt.	Smart plug'en skifter tilstand uden brug af cloud.	Routerens internetforbindelse blev afbrudt, mens det lokale netværk var aktivt. Smart plug'en tændte og slukkede som normalt, hvilket viser, at styringen sker lokalt uden brug af cloud.	Bestået
K-008	Internetforbindelsen afbrydes efter, at dagens priser er hentet.	Styringen fortsætter med de gemte priser.	Internetforbindelsen blev afbrudt, efter at dagens priser var hentet. Styringen fortsatte med de gemte priser, og beslutningerne blev truffet ud fra prisgrænsen på 1,0843 DKK/kWh.	Bestået
K-009	Efter en testperiode åbnes loggen.	Hver tænding og slukning står med tidspunkt.	Loggen viser hver tænding og slukning med tidspunkt og begrundelse, bl.a. tidspunkt,	Bestået

Krav-ID	Testhandling	Forventet resultat	Faktisk resultat	Status
			luftfugtighed, elpris og prisgrænse.	
K-010	Efter en testperiode åbnes databasen.	Hver måling står med tidspunkt.	Databasen indeholdt 34102 målinger fra testperioden, alle med tidspunkt og kilde.	Bestået
K-011	Internettet afbrydes, og der findes ingen gemte priser.	Sikkerhedstilstand: kritisk grænse og slukgrænse gælder, vises på websiden.	Med internettet afbrudt og uden gemte priser gik systemet i sikkerhedstilstand. Affugteren blev kun tændt ved luftfugtighed over 70 % RH og slukket ved 45 % RH. Fejlen blev vist på websiden.	Bestået
K-012	Begge sensorer frakobles, mens affugteren kører, og tilsluttes igen.	Når målingen er forældet, slukkes affugteren, og fejlen logges. Normal styring genoptages ved næste styringscyklus efter gentilslutning.	Sensoren blev frakoblet kl. 12.21. Efter 5 mislykkede aflæsninger i træk blev affugteren slukket, og fejlen blev logget som RuntimeError ("DHT11 failed 5 times in a row. Shutting off the plug.") og vist på websiden. Efter gentilslutning blev normal styring genoptaget ved næste styringscyklus.	Bestået
K-013	Smart plug'en frakobles netværket og tilsluttes igen.	Fejlen logges, og styringen fortsætter ved næste styringscyklus efter gentilslutning.	Smart plug'en blev frakoblet kl. 10.24. Fejlen blev logget i databasen som ConnectTimeout og	Bestået

Krav-ID	Testhandling	Forventet resultat	Faktisk resultat	Status
			ConnectError ("All connection attempts failed") og vist på websiden, hvor affugterens status skiftede til "Offline – Shelly svarer ikke". Efter gentilslutning viste siden igen korrekt status ("SLUKKET"), og styringen fortsatte ved næste styringscyklus.	
K-014	Strømmen til Raspberry Pi'en afbrydes og tilsluttes igen.	Systemet starter selv og genoptager styringen uden brugerens hjælp. Affugteren tændes tidligst efter 15 minutter.	Strømmen til Raspberry Pi'en blev afbrudt og tilsluttet igen. Efter opstart viste <i>systemctl is-active affugter</i> statussen "active", og systemet havde dermed startet applikationen automatisk og genoptaget styringen uden brugerens hjælp.	Bestået
K-015	Programmet stoppes manuelt (fx med kill).	Programmet genstartes automatisk inden for [30] sekunder.	Programmet blev stoppet hårdt med <i>pkill -9 -f uvicorn</i> . 15 sekunder senere viste <i>systemctl is-active affugter</i> statussen "active", og tjenesten var dermed genstartet automatisk af systemd.	Bestået

Krav-ID	Testhandling	Forventet resultat	Faktisk resultat	Status
K-016	Websiden åbnes fra en enhed på det lokale netværk.	Siden viser luftfugtighed, tænd/sluk, prisgrænse og aktive fejl, og værdierne svarer til de faktiske.	Websiden blev åbnet fra en enhed på det lokale netværk kl. 10.46 og viste luftfugtighed 64,0 % RH, temperatur 23,8 °C, affugteren slukket, aktuel elpris 1,1462 DKK/kWh og prisgrænsen 0,8912 DKK/kWh. Værdierne svarede til de nyeste rækker i databasen. Fejl blev vist under "Seneste systemfejl".	Bestået
K-017	Systemet kører i [72] timer uden indgriben.	Loggen viser målinger og beslutninger i hele perioden uden manuelle genstarter.	Loggen dækker perioden 2.–6. oktober med beslutninger hver dag og ingen manuelle genstarter.	Bestået
K-018	(a) Affugteren kører kl. 21.59. (b) Luftfugtigheden hæves over den kritiske grænse kl. 23.00.	(a) Smart plug'en slukkes kl. 22.00. (b) Smart plug'en forbliver slukket til kl. 06.00.	Den 5. oktober kl. 21.45 blev affugteren tændt ved 70,9 % RH. Kl. 22.00 blev den slukket med begrundelsen "Dark time active", og den forblev slukket til kl. 06.00, hvor den blev tændt igen ved 74,9 % RH.	Bestået

6. Konklusion

Det er lykkedes os at fremstille et system, som dækker de krav, der er stillet til produktet. Systemet måler via enten DHT11- eller ZigBee-sensor rummets aktuelle luftfugtighed. Dette

giver adgang til basal styring, som kan ses ved at affugteren tændes, selv uden elprisdata, hvis luftfugtigheden bliver kritisk høj.

Systemet indhenter automatisk elprisdata fra Energidataservice-API'et. Disse data bruges sammen med sensormålingerne til at beslutte, hvornår affugteren skal være tændt eller slukket.

Hvis den målte luftfugtighed overstiger den kritiske grænse på 70%, bliver affugteren tændt, uanset elpris. Dette sørger for, at rummet altid holdes indenfor den for brugeren acceptable grænse for luftfugtighed.

Det er et krav, at der tages højde for affugterens kompressor, for at maskinen kører optimalt. Derfor er der, sensor- og elprisdata uagtet, indsat et kompressor-lockout, så affugteren aldrig kan tændes indenfor 15 minutter af en tidligere slukning.

Automatisering er fundamentalt for denne løsning. For at undgå afhængighed af en ekstern server eller service, kører systemet lokalt i hjemmet. Smart plug og Raspberry Pi er begge forbundet lokalt på et netværk, og all kommunikation mellem dem foregår lokalt. Der er derfor ikke mulighed for reel fjernstyring uden en privat VPN-service, men det fjerner afhængigheden af endnu en ekstern leverandør.

For at systemet kan fungere uden brugerovervågning, er der implementeret gennemgående fejlhåndtering. Systemet fungerer med nedsat funktionalitet uden internetforbindelse, or derfor elprisdata. Ved fejl på sensor forsøger systemet løbende at læse videre på sensoren, mens fejl logges. Det samme gælder for kommunikation med smart plug'en. Systemet fungerer ikke uden database eller sensordata.

7. Bilag

Bilag 1: Kravanalyse – kravkandidater og foreløbige krav

Bilag 2: Datablad for affugteren (eeese Emil 2508)

Bilag 3: API-oversigt (vedlagt som separat fil)